

4. Check whether pinning is allowed

Pinning to the taskbar can be disabled by Group Policy. The [TaskbarManager.IsPinningAllowed](#) property lets you check whether pinning is allowed.

When the user clicks your pin button, you should check this property, and if it's false, you should display a message dialog informing the user that pinning is not allowed on this machine.

C#

```
TaskbarManager taskbarManager = TaskbarManager.GetDefault();
if (taskbarManager == null)
{
    // Display message dialog informing user that taskbar is no longer
    // present, and then hide the button
}

else if (taskbarManager.IsPinningAllowed == false)
{
    // Display message dialog informing user pinning is not allowed on this
    // machine
}

else
{
    // Continue pinning
}
```

5. Construct and pin your tile

The user has clicked your pin button, and you've determined that the APIs are present, taskbar is present, and pinning is allowed... time to pin!

First, construct your secondary tile just like you would when pinning to Start. You can learn more about the secondary tile properties by reading [Pin secondary tiles to Start](#). However, when pinning to taskbar, in addition to the previously required properties, `Square44x44Logo` (this is the logo used by taskbar) is also required. Otherwise, an exception will be thrown.

Then, pass the tile to the [RequestPinSecondaryTileAsync](#) method. Since this is under limited-access, this will not display a confirmation dialog and does not require a UI thread. But in the future when this is opened up beyond limited-access, callers not utilizing limited-access will receive a dialog and be required to use the UI thread.

C#

```
// Initialize the tile (all properties below are required)
SecondaryTile tile = new SecondaryTile("myTileId");
tile.DisplayName = "PowerPoint 2016 (Remote)";
tile.Arguments = "app=powerpoint";
tile.VisualElements.Square44x44Logo = new Uri("ms-appdata:///AppIcons/PowerPoint_Square44x44Logo.png");
tile.VisualElements.Square150x150Logo = new Uri("ms-appdata:///AppIcons/PowerPoint_Square150x150Logo.png");

// Pin it to the taskbar
bool isPinned = await taskbarManager.RequestPinSecondaryTileAsync(tile);
```

This method returns a boolean value that indicates whether your tile is now pinned to the taskbar. If your tile was already pinned, the method updates the existing tile and returns `true`. If pinning wasn't allowed or taskbar isn't supported, the method returns `false`.

Enumerate tiles

To see all the tiles that you created and are still pinned somewhere (Start, taskbar, or both), use [FindAllAsync](#). You can subsequently check whether these tiles are pinned to the taskbar and/or Start. If the surface isn't supported, these methods return false.

C#

```
var taskbarManager = TaskbarManager.Default;
var startScreenManager = StartScreenManager.Default;

// Look through all tiles
foreach (SecondaryTile tile in await SecondaryTile.FindAllAsync())
{
    if (taskbarManager != null && await
taskbarManager.IsSecondaryTilePinnedAsync(tile.TileId))
    {
        // Tile is pinned to the taskbar
    }

    if (startScreenManager != null && await
startScreenManager.ContainsSecondaryTileAsync(tile.TileId))
    {
        // Tile is pinned to Start
    }
}
```

Update a tile

To update an already pinned tile, you can use the [SecondaryTile.UpdateAsync](#) method as described in [Updating a secondary tile](#).

Unpin a tile

Your app should provide an unpin button if the tile is currently pinned. To unpin the tile, simply call [TryUnpinSecondaryTileAsync](#), passing in the `TileId` of the secondary tile you would like unpinned.

This method returns a boolean value that indicates whether your tile is no longer pinned to the taskbar. If your tile wasn't pinned in the first place, this also returns `true`. If unpinning wasn't allowed, this returns `false`.

If your tile was only pinned to taskbar, this will delete the tile since it is no longer pinned anywhere.

C#

```
var taskbarManager = TaskbarManager.Default();
if (taskbarManager != null)
{
    bool isUnpinned = await
taskbarManager.TryUnpinSecondaryTileAsync("myTileId");
}
```

Delete a tile

If you want to unpin a tile from everywhere (Start, taskbar), use the [RequestDeleteAsync](#) method.

This is appropriate for cases where the content the user pinned is no longer applicable. For example, if your app lets you pin a notebook to Start and taskbar, and then the user deletes the notebook, you should simply delete the tile associated with the notebook.

C#

```
// Initialize a secondary tile with the same tile ID you want removed.
// Or, locate it with FindAllAsync()
SecondaryTile toBeDeleted = new SecondaryTile(tileId);

// And then delete the tile
await toBeDeleted.RequestDeleteAsync();
```

Unpin only from Start

If you only want to unpin a secondary tile from Start while leaving it on Taskbar, you can call the [StartScreenManager.TryRemoveSecondaryTileAsync](#) method. This will similarly delete the tile if it is no longer pinned to any other surfaces.

This method returns a boolean value that indicates whether your tile is no longer pinned to Start. If your tile wasn't pinned in the first place, this also returns `true`. If unpinning wasn't allowed or Start isn't supported, this returns `false`.

C#

```
await  
StartScreenManager.Default().TryRemoveSecondaryTileAsync("myTileId");
```

Resources

- [TaskbarManager class](#)
- [Pin secondary tiles to Start](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create adaptive tiles

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Adaptive tile templates are a new feature in Windows 10, allowing you to design your own tile notification content using a simple and flexible markup language that adapts to different screen densities. This article tells you how to create adaptive live tiles for your Windows app. For the complete list of adaptive elements and attributes, see the [Adaptive tiles schema](#).

(If you'd like, you can still use the preset templates from the [Windows 8 tile template catalog](#) when designing notifications for Windows 10.)

Getting started

Install Notifications library. If you'd like to use C# instead of XML to generate notifications, install the NuGet package named [Microsoft.Toolkit.Uwp.Notifications](#) [↗](#) (search for "notifications uwp"). The C# samples provided in this article use version 1.0.0 of the NuGet package.

Install Notifications Visualizer. This free Windows app helps you design adaptive live tiles by providing an instant visual preview of your tile as you edit it, similar to Visual Studio's XAML editor/design view. See [Notifications Visualizer](#) for more information, or [download Notifications Visualizer from the Store](#) [↗](#).

How to send a tile notification

Please read our [Quickstart on sending local tile notifications](#). The documentation below explains all the visual UI possibilities you have with adaptive tiles.

Usage guidance

Adaptive templates are designed to work across different form factors and notification types. Elements such as group and subgroup link together content and don't imply a

particular visual behavior on their own. The final appearance of a notification should be based on the specific device on which it will appear, whether it's phone, tablet, or desktop, or another device.

Hints are optional attributes that can be added to elements in order to achieve a specific visual behavior. Hints can be device-specific or notification-specific.

A basic example

This example demonstrates what the adaptive tile templates can produce.

XML

```
<tile>
  <visual>

    <binding template="TileMedium">
      ...
    </binding>

    <binding template="TileWide">
      <text hint-style="subtitle">Jennifer Parker</text>
      <text hint-style="captionSubtitle">Photos from our trip</text>
      <text hint-style="captionSubtitle">Check out these awesome photos I took
while in New Zealand!</text>
    </binding>

    <binding template="TileLarge">
      ...
    </binding>

  </visual>
</tile>
```

C#

```
TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        TileMedium = ...

        TileWide = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                Children =
                {
                    new AdaptiveText()
                    {

```

```

        Text = "Jennifer Parker",
        HintStyle = AdaptiveTextStyle.Subtitle
    },

    new AdaptiveText()
    {
        Text = "Photos from our trip",
        HintStyle = AdaptiveTextStyle.CaptionSubtle
    },

    new AdaptiveText()
    {
        Text = "Check out these awesome photos I took while
in New Zealand!",
        HintStyle = AdaptiveTextStyle.CaptionSubtle
    }
    },
    TileLarge = ...
}
};

```

Result:



Tile sizes

Content for each tile size is individually specified in separate [TileBinding](#) elements within the XML payload. Choose the target size by setting the template attribute to one of the following values:

- `TileSmall`
- `TileMedium`
- `TileWide`
- `TileLarge`

For a single tile notification XML payload, provide `<binding>` elements for each tile size that you'd like to support, as shown in this example:

XML

```

<tile>
  <visual>

    <binding template="TileSmall">
      <text>Small</text>
    </binding>

    <binding template="TileMedium">
      <text>Medium</text>
    </binding>

    <binding template="TileWide">
      <text>Wide</text>
    </binding>

    <binding template="TileLarge">
      <text>Large</text>
    </binding>

  </visual>
</tile>

```

C#

```

TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        TileSmall = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                Children =
                {
                    new AdaptiveText() { Text = "Small" }
                }
            }
        },

        TileMedium = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                Children =
                {
                    new AdaptiveText() { Text = "Medium" }
                }
            }
        },

        TileWide = new TileBinding()
        {

```

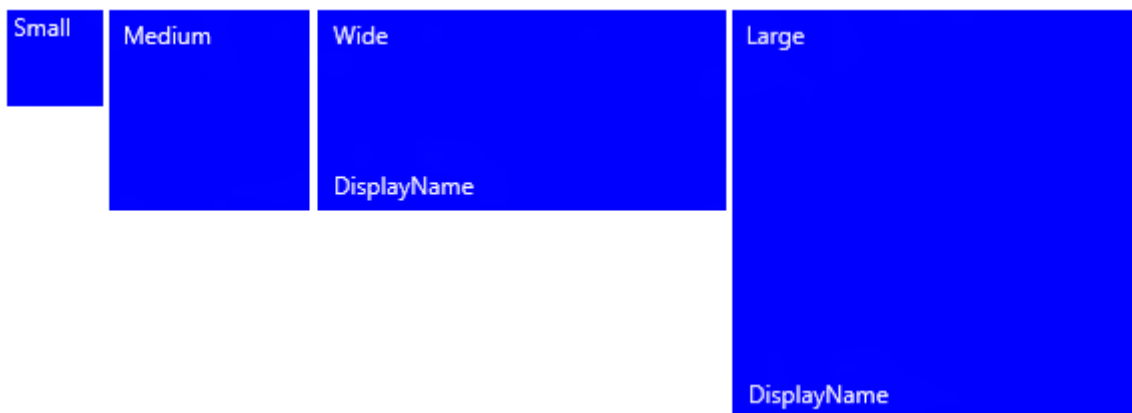


```

        Content = new TileBindingContentAdaptive()
        {
            Children =
            {
                new AdaptiveText() { Text = "Wide" }
            }
        },
        TileLarge = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                Children =
                {
                    new AdaptiveText() { Text = "Large" }
                }
            }
        }
    }
};

```

Result:



Branding

You can control the branding on the bottom of a live tile (the display name and corner logo) by using the branding attribute on the notification payload. You can choose to display `none`, only the `name`, only the `logo`, or both with `nameAndLogo`.

XML

```

<visual branding="logo">
    ...
</visual>

```

C#

```
new TileVisual()  
{  
    Branding = TileBranding.Logo,  
    ...  
}
```

Result:



Branding can be applied for specific tile sizes one of two ways:

1. By applying the attribute on the [TileBinding](#) element.
2. By applying the attribute on the [TileVisual](#) element, which affects the entire notification payload.

If you don't specify branding for a binding, it will use the branding that's provided on the visual element.

XML

```
<tile>  
  <visual branding="nameAndLogo">  
  
    <binding template="TileMedium" branding="logo">  
      ...  
    </binding>  
  
    <!--Inherits branding from visual-->  
    <binding template="TileWide">  
      ...  
    </binding>  
  
  </visual>  
</tile>
```

C#

```
TileContent content = new TileContent()  
{  
    Visual = new TileVisual()  
    {
```

```

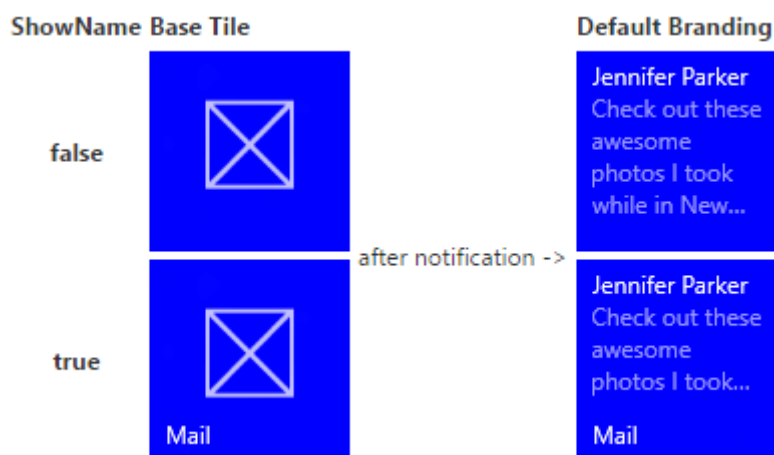
        Branding = TileBranding.NameAndLogo,

        TileMedium = new TileBinding()
        {
            Branding = TileBranding.Logo,
            ...
        },

        // Inherits branding from Visual
        TileWide = new TileBinding()
        {
            ...
        }
    }
};

```

Default branding result:



If you don't specify the branding in your notification payload, the base tile's properties will determine the branding. If the base tile shows the display name, then the branding will default to `name`. Otherwise, the branding will default to `none` if the display name isn't shown.

Note This is a change from Windows 8.x, in which the default branding was "logo."

Display name

You can override the display name of a notification by entering the text string of your choice with the `displayName` attribute. As with branding, you can specify this on the [TileVisual](#) element, which affects the entire notification payload, or on the [TileBinding](#) element, which only affects individual tiles.

XML

```

<tile>
  <visual branding="nameAndLogo" displayName="Wednesday 22">

```

```

<binding template="TileMedium" displayName="Wed. 22">
    ...
</binding>

<!--Inherits displayName from visual-->
<binding template="TileWide">
    ...
</binding>

</visual>
</tile>

```

C#

```

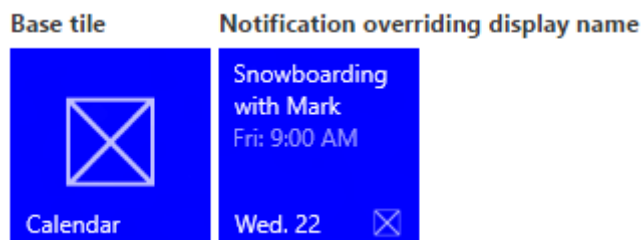
TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        Branding = TileBranding.NameAndLogo,
        DisplayName = "Wednesday 22",

        TileMedium = new TileBinding()
        {
            DisplayName = "Wed. 22",
            ...
        },

        // Inherits DisplayName from Visual
        TileWide = new TileBinding()
        {
            ...
        }
    }
};

```

Result:



Text

The [AdaptiveText](#) element is used to display text. You can use hints to modify how text appears.

XML

```
<text>This is a line of text</text>
```

C#

```
new AdaptiveText()  
{  
    Text = "This is a line of text"  
};
```

Result:



Text wrapping

By default, text doesn't wrap and will continue off the edge of the tile. Use the `hint-wrap` attribute to set text wrapping on a text element. You can also control the minimum and maximum number of lines by using `hint-minLines` and `hint-maxLines`, both of which accept positive integers.

XML

```
<text hint-wrap="true">This is a line of wrapping text</text>
```

C#

```
new AdaptiveText()  
{  
    Text = "This is a line of wrapping text",  
    HintWrap = true  
};
```

Result:

This is a line of
wrapping text

DisplayName

Text styles

Styles control the font size, color, and weight of text elements. There are a number of available styles, including a "subtle" variation of each style that sets the opacity to 60%, which usually makes the text color a shade of light gray.

XML

```
<text hint-style="base">Header content</text>
<text hint-style="captionSubtle">Subheader content</text>
```

C#

```
new AdaptiveText()
{
    Text = "Header content",
    HintStyle = AdaptiveTextStyle.Base
},

new AdaptiveText()
{
    Text = "Subheader content",
    HintStyle = AdaptiveTextStyle.CaptionSubtle
}
```

Result:

Header content

Subheader content

DisplayName

Note The style defaults to caption if hint-style isn't specified.

Basic text styles

[Expand table](#)

<code><text hint-style="*" /></code>	Font height	Font weight
caption	12 effective pixels (epx)	Regular
body	15 epx	Regular
base	15 epx	Semibold
subtitle	20 epx	Regular
title	24 epx	Semilight
subheader	34 epx	Light
header	46 epx	Light

Numeral text style variations

These variations reduce the line height so that content above and below come much closer to the text.

- titleNumeral
- subheaderNumeral
- headerNumeral

Subtle text style variations

Each style has a subtle variation that gives the text a 60% opacity, which usually makes the text color a shade of light gray.

- captionSubtle
- bodySubtle
- baseSubtle
- subtitleSubtle
- titleSubtle
- titleNumeralSubtle
- subheaderSubtle
- subheaderNumeralSubtle
- headerSubtle
- headerNumeralSubtle

Text alignment

Text can be horizontally aligned left, center, or right. In left-to-right languages like English, text defaults to left-aligned. In right-to-left languages like Arabic, text defaults

to right-aligned. You can manually set alignment with the **hint-align** attribute on elements.

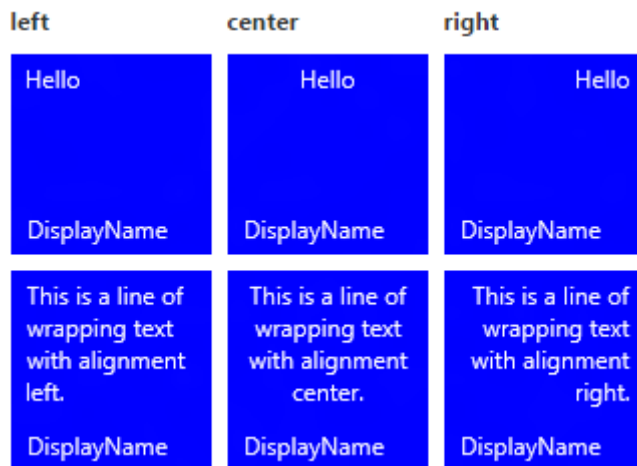
XML

```
<text hint-align="center">Hello</text>
```

C#

```
new AdaptiveText()  
{  
    Text = "Hello",  
    HintAlign = AdaptiveTextAlign.Center  
};
```

Result:



Groups and subgroups

Groups allow you to semantically declare that the content inside the group is related and must be displayed in its entirety for the content to make sense. For example, you might have two text elements, a header, and a subheader, and it would not make sense for only the header to be shown. By grouping those elements inside a subgroup, the elements will either all be displayed (if they can fit) or not be displayed at all (because they can't fit).

To provide the best experience across devices and screens, provide multiple groups. Having multiple groups allows your tile to adapt to larger screens.

Note The only valid child of a group is a subgroup.

XML


```

<binding template="TileWide" branding="nameAndLogo">
  <group>
    <subgroup>
      <text hint-style="subtitle">Jennifer Parker</text>
      <text hint-style="captionSubtitle">Photos from our trip</text>
      <text hint-style="captionSubtitle">Check out these awesome photos I took
while in New Zealand!</text>
    </subgroup>
  </group>

  <text />

  <group>
    <subgroup>
      <text hint-style="subtitle">Steve Bosniak</text>
      <text hint-style="captionSubtitle">Build 2015 Dinner</text>
      <text hint-style="captionSubtitle">Want to go out for dinner after Build
tonight?</text>
    </subgroup>
  </group>
</binding>

```

C#

```

TileWide = new TileBinding()
{
    Branding = TileBranding.NameAndLogo,
    Content = new TileBindingContentAdaptive()
    {
        Children =
        {
            CreateGroup(
                from: "Jennifer Parker",
                subject: "Photos from our trip",
                body: "Check out these awesome photos I took while in New
Zealand!"),

            // For spacing
            new AdaptiveText(),

            CreateGroup(
                from: "Steve Bosniak",
                subject: "Build 2015 Dinner",
                body: "Want to go out for dinner after Build tonight?")
        }
    }
}

...

private static AdaptiveGroup CreateGroup(string from, string subject, string
body)

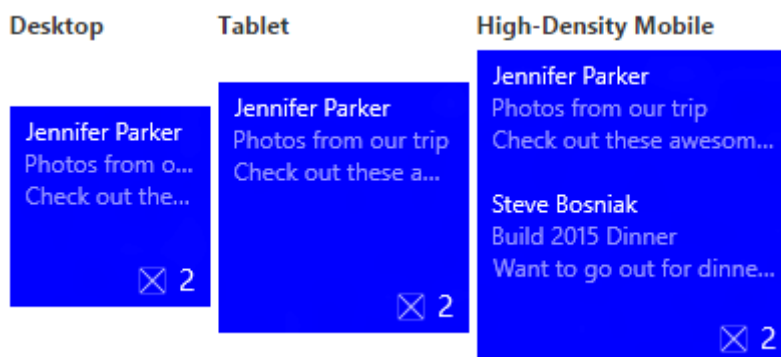
```

```

{
    return new AdaptiveGroup()
    {
        Children =
        {
            new AdaptiveSubgroup()
            {
                Children =
                {
                    new AdaptiveText()
                    {
                        Text = from,
                        HintStyle = AdaptiveTextStyle.Subtitle
                    },
                    new AdaptiveText()
                    {
                        Text = subject,
                        HintStyle = AdaptiveTextStyle.CaptionSubtle
                    },
                    new AdaptiveText()
                    {
                        Text = body,
                        HintStyle = AdaptiveTextStyle.CaptionSubtle
                    }
                }
            }
        }
    };
}

```

Result:



Subgroups (columns)

Subgroups also allow you to divide data into semantic sections within a group. For live tiles, this visually translates to columns.

The `hint-weight` attribute lets you to control the widths of columns. The value of `hint-weight` is expressed as a weighted proportion of available space, which is identical to

`GridUnitType.Star` behavior. For equal-width columns, assign each weight to 1.

[Expand table](#)

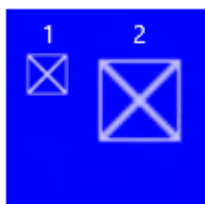
hint-weight	Percentage of width
1	25%
1	25%
1	25%
1	25%
Total weight: 4	



To make one column twice as large as another column, assign the smaller column a weight of 1 and the larger column a weight of 2.

[Expand table](#)

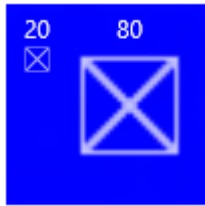
hint-weight	Percentage of width
1	33.3%
2	66.7%
Total weight: 3	



If you want your first column to take up 20% of the total width and your second column to take up 80% of the total width, assign the first weight to 20 and the second weight to 80. If your total weights equal 100, they'll act as percentages.

[Expand table](#)

hint-weight	Percentage of width
20	20%
80	80%
Total weight: 100	



Note An 8-pixel margin is automatically added between the columns.

When you have more than two subgroups, you should specify the `hint-weight`, which only accepts positive integers. If you don't specify `hint-weight` for the first subgroup, it will be assigned a weight of 50. The next subgroup that doesn't have a specified `hint-weight` will be assigned a weight equal to 100 minus the sum of the preceding weights, or to 1 if the result is zero. The remaining subgroups that don't have specified hint-weights will be assigned a weight of 1.

Here's sample code for a weather tile that shows how you can achieve a tile with five columns of equal width:

XML

```
<binding template="TileWide" displayName="Seattle" branding="name">
  <group>
    <subgroup hint-weight="1">
      <text hint-align="center">Mon</text>
      <image src="Assets\Weather\Mostly Cloudy.png" hint-
removeMargin="true"/>
      <text hint-align="center">63°</text>
      <text hint-align="center" hint-style="captionsubtle">42°</text>
    </subgroup>
    <subgroup hint-weight="1">
      <text hint-align="center">Tue</text>
      <image src="Assets\Weather\Cloudy.png" hint-removeMargin="true"/>
      <text hint-align="center">57°</text>
      <text hint-align="center" hint-style="captionsubtle">38°</text>
    </subgroup>
    <subgroup hint-weight="1">
      <text hint-align="center">Wed</text>
      <image src="Assets\Weather\Sunny.png" hint-removeMargin="true"/>
      <text hint-align="center">59°</text>
      <text hint-align="center" hint-style="captionsubtle">43°</text>
    </subgroup>
  </group>
</binding>
```

```

<subgroup hint-weight="1">
  <text hint-align="center">Thu</text>
  <image src="Assets\Weather\Sunny.png" hint-removeMargin="true"/>
  <text hint-align="center">62°</text>
  <text hint-align="center" hint-style="captionsubtle">42°</text>
</subgroup>
<subgroup hint-weight="1">
  <text hint-align="center">Fri</text>
  <image src="Assets\Weather\Sunny.png" hint-removeMargin="true"/>
  <text hint-align="center">71°</text>
  <text hint-align="center" hint-style="captionsubtle">66°</text>
</subgroup>
</group>
</binding>

```

C#

```

TileWide = new TileBinding()
{
    DisplayName = "Seattle",
    Branding = TileBranding.Name,
    Content = new TileBindingContentAdaptive()
    {
        Children =
        {
            new AdaptiveGroup()
            {
                Children =
                {
                    CreateSubgroup("Mon", "Mostly Cloudy.png", "63°",
"42°"),
                    CreateSubgroup("Tue", "Cloudy.png", "57°", "38°"),
                    CreateSubgroup("Wed", "Sunny.png", "59°", "43°"),
                    CreateSubgroup("Thu", "Sunny.png", "62°", "42°"),
                    CreateSubgroup("Fri", "Sunny.png", "71°", "66°")
                }
            }
        }
    }
}

...

private static AdaptiveSubgroup CreateSubgroup(string day, string image,
string highTemp, string lowTemp)
{
    return new AdaptiveSubgroup()
    {
        HintWeight = 1,
        Children =
        {
            new AdaptiveText()
            {

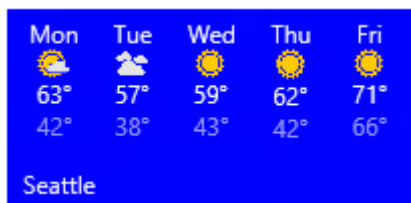
```

```

        Text = day,
        HintAlign = AdaptiveTextAlign.Center
    },
    new AdaptiveImage()
    {
        Source = "Assets/Weather/" + image,
        HintRemoveMargin = true
    },
    new AdaptiveText()
    {
        Text = highTemp,
        HintAlign = AdaptiveTextAlign.Center
    },
    new AdaptiveText()
    {
        Text = lowTemp,
        HintAlign = AdaptiveTextAlign.Center,
        HintStyle = AdaptiveTextStyle.CaptionSubtle
    }
    }
};
}

```

Result:



Images

The `<image>` element is used to display images on the tile notification. Images can be placed inline within the tile content (default), as a background image behind your content, or as a peek image that animates in from the top of the notification.

ⓘ Note

Images can be used from the app's package, the app's local storage, or from the web. As of the Fall Creators Update, web images can be up to 3 MB on normal connections and 1 MB on metered connections. On devices not yet running the Fall Creators Update, web images must be no larger than 200 KB.

With no extra behaviors specified, images will uniformly shrink or expand to fill the available width. This example shows a tile using two columns and inline images. The

inline images stretch to fill the width of the column.

XML

```
<binding template="TileMedium" displayName="Seattle" branding="name">
  <group>
    <subgroup>
      <text hint-align="center">Mon</text>
      <image src="Assets\Apps\Weather\Mostly Cloudy.png" hint-
removeMargin="true"/>
      <text hint-align="center">63°</text>
      <text hint-style="captionSubtle" hint-align="center">42°</text>
    </subgroup>
    <subgroup>
      <text hint-align="center">Tue</text>
      <image src="Assets\Apps\Weather\Cloudy.png" hint-removeMargin="true"/>
      <text hint-align="center">57°</text>
      <text hint-style="captionSubtle" hint-align="center">38°</text>
    </subgroup>
  </group>
</binding>
```

C#

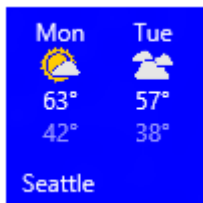
```
TileMedium = new TileBinding()
{
    DisplayName = "Seattle",
    Branding = TileBranding.Name,
    Content = new TileBindingContentAdaptive()
    {
        Children =
        {
            new AdaptiveGroup()
            {
                Children =
                {
                    CreateSubgroup("Mon", "Mostly Cloudy.png", "63°",
"42°"),
                    CreateSubgroup("Tue", "Cloudy.png", "57°", "38°")
                }
            }
        }
    }
}
...
private static AdaptiveSubgroup CreateSubgroup(string day, string image,
string highTemp, string lowTemp)
{
    return new AdaptiveSubgroup()
    {
        Children =
        {
            new AdaptiveText()
```

```

        {
            Text = day,
            HintAlign = AdaptiveTextAlign.Center
        },
        new AdaptiveImage()
        {
            Source = "Assets/Weather/" + image,
            HintRemoveMargin = true
        },
        new AdaptiveText()
        {
            Text = highTemp,
            HintAlign = AdaptiveTextAlign.Center
        },
        new AdaptiveText()
        {
            Text = lowTemp,
            HintAlign = AdaptiveTextAlign.Center,
            HintStyle = AdaptiveTextStyle.CaptionSubtle
        }
    }
};
}

```

Result:



Images placed in the <binding> root, or in the first group, will also stretch to fit the available height.

Image alignment

Images can be set to align left, center, or right using the **hint-align** attribute. This will also cause images to display at their native resolution instead of stretching to fill width.

XML

```

<binding template="TileLarge">
    <image src="Assets/fable.jpg" hint-align="center"/>
</binding>

```

C#


```

TileLarge = new TileBinding()
{
    Content = new TileBindingContentAdaptive()
    {
        Children =
        {
            new AdaptiveImage()
            {
                Source = "Assets/fable.jpg",
                HintAlign = AdaptiveImageAlign.Center
            }
        }
    }
}

```

Result:

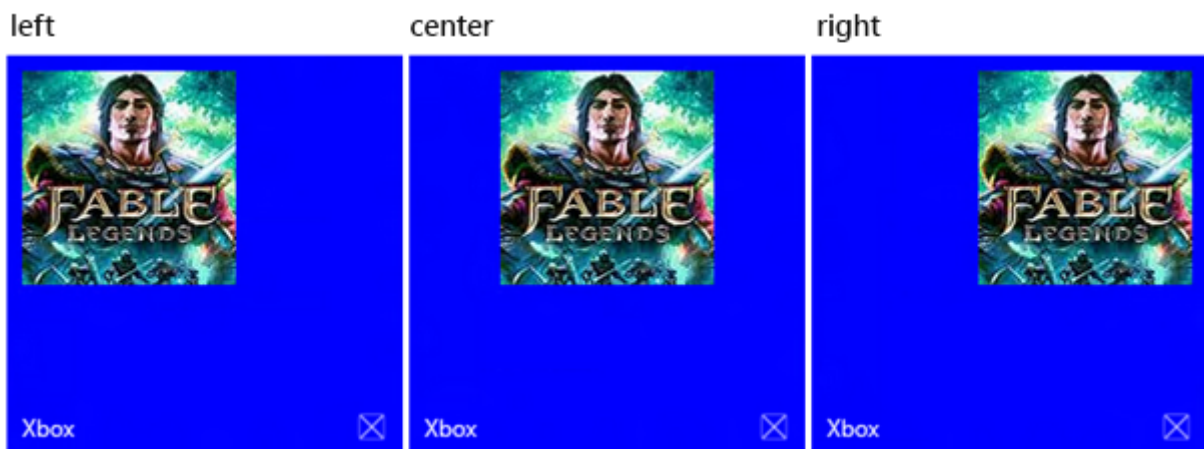


Image margins

By default, inline images have an 8-pixel margin between any content above or below the image. This margin can be removed by using the **hint-removeMargin** attribute on the image. However, images always retain the 8-pixel margin from the edge of the tile, and subgroups (columns) always retain the 8-pixel padding between columns.

XML

```

<binding template="TileMedium" branding="none">
  <group>
    <subgroup>
      <text hint-align="center">Mon</text>
      <image src="Assets\Numbers\4.jpg" hint-removeMargin="true"/>
      <text hint-align="center">63°</text>
      <text hint-style="captionsubtle" hint-align="center">42°</text>
    </subgroup>
    <subgroup>
      <text hint-align="center">Tue</text>
    </subgroup>
  </group>
</binding>

```

```

<image src="Assets\Numbers\3.jpg" hint-removeMargin="true"/>
<text hint-align="center">57°</text>
<text hint-style="captionsubtle" hint-align="center">38°</text>
</subgroup>
</group>
</binding>

```

C#

```

TileMedium = new TileBinding()
{
    Branding = TileBranding.None,
    Content = new TileBindingContentAdaptive()
    {
        Children =
        {
            new AdaptiveGroup()
            {
                Children =
                {
                    CreateSubgroup("Mon", "4.jpg", "63°", "42°"),
                    CreateSubgroup("Tue", "3.jpg", "57°", "38°")
                }
            }
        }
    }
}

...

private static AdaptiveSubgroup CreateSubgroup(string day, string image,
string highTemp, string lowTemp)
{
    return new AdaptiveSubgroup()
    {
        HintWeight = 1,
        Children =
        {
            new AdaptiveText()
            {
                Text = day,
                HintAlign = AdaptiveTextAlign.Center
            },
            new AdaptiveImage()
            {
                Source = "Assets/Numbers/" + image,
                HintRemoveMargin = true
            },
            new AdaptiveText()
            {
                Text = highTemp,
                HintAlign = AdaptiveTextAlign.Center
            },

```

```

        new AdaptiveText()
        {
            Text = lowTemp,
            HintAlign = AdaptiveTextAlign.Center,
            HintStyle = AdaptiveTextStyle.CaptionSubtle
        }
    };
}

```

hint-removeMargin

false

true



Image cropping

Images can be cropped into a circle using the **hint-crop** attribute, which currently only supports the values "none" (default) or "circle."

XML

```

<binding template="TileLarge" hint-textStacking="center">
    <group>
        <subgroup hint-weight="1"/>
        <subgroup hint-weight="2">
            <image src="Assets/Apps/Hipstame/hipster.jpg" hint-crop="circle"/>
        </subgroup>
        <subgroup hint-weight="1"/>
    </group>

    <text hint-style="title" hint-align="center">Hi,</text>
    <text hint-style="subtitleSubtle" hint-align="center">MasterHip</text>
</binding>

```

C#

```

TileLarge = new TileBinding()
{
    Content = new TileBindingContentAdaptive()
    {
        TextStacking = TileTextStacking.Center,
        Children =
        {

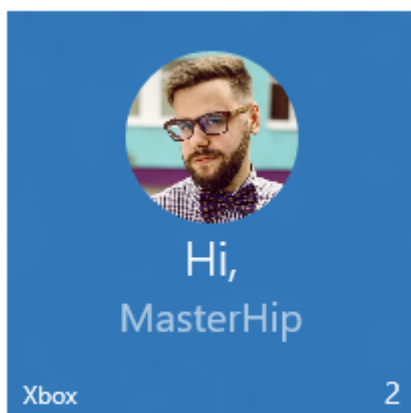
```

```

new AdaptiveGroup()
{
    Children =
    {
        new AdaptiveSubgroup() { HintWeight = 1 },
        new AdaptiveSubgroup()
        {
            HintWeight = 2,
            Children =
            {
                new AdaptiveImage()
                {
                    Source = "Assets/Apps/Hipstame/hipster.jpg",
                    HintCrop = AdaptiveImageCrop.Circle
                }
            }
        },
        new AdaptiveSubgroup() { HintWeight = 1 }
    }
},
new AdaptiveText()
{
    Text = "Hi,",
    HintStyle = AdaptiveTextStyle.Title,
    HintAlign = AdaptiveTextAlign.Center
},
new AdaptiveText()
{
    Text = "MasterHip",
    HintStyle = AdaptiveTextStyle.SubtitleSubtle,
    HintAlign = AdaptiveTextAlign.Center
}
}
}
}

```

Result:



Background image

To set a background image, place an image element in the root of the <binding> and set the placement attribute to "background."

XML

```
<binding template="TileWide">
  <image src="Assets\Mostly Cloudy-Background.jpg" placement="background"/>
  <group>
    <subgroup hint-weight="1">
      <text hint-align="center">Mon</text>
      <image src="Assets\Weather\Mostly Cloudy.png" hint-
removeMargin="true"/>
      <text hint-align="center">63°</text>
      <text hint-align="center" hint-style="captionsubtle">42°</text>
    </subgroup>
    ...
  </group>
</binding>
```

C#

```
TileWide = new TileBinding()
{
    Content = new TileBindingContentAdaptive()
    {
        BackgroundImage = new TileBackgroundImage()
        {
            Source = "Assets/Mostly Cloudy-Background.jpg"
        },
        Children =
        {
            new AdaptiveGroup()
            {
                Children =
                {
                    CreateSubgroup("Mon", "Mostly Cloudy.png", "63°", "42°")
                    ...
                }
            }
        }
    }
}

...

private static AdaptiveSubgroup CreateSubgroup(string day, string image,
string highTemp, string lowTemp)
{
    return new AdaptiveSubgroup()
    {
        HintWeight = 1,
```

```

Children =
{
    new AdaptiveText()
    {
        Text = day,
        HintAlign = AdaptiveTextAlign.Center
    },
    new AdaptiveImage()
    {
        Source = "Assets/Weather/" + image,
        HintRemoveMargin = true
    },
    new AdaptiveText()
    {
        Text = highTemp,
        HintAlign = AdaptiveTextAlign.Center
    },
    new AdaptiveText()
    {
        Text = lowTemp,
        HintAlign = AdaptiveTextAlign.Center,
        HintStyle = AdaptiveTextStyle.CaptionSubtle
    }
}
};
}

```

Result:



Peek image

You can specify an image that "peeks" in from the top of the tile. The peek image uses an animation to slide down/up from the top of the tile, peeking into view, and then later sliding back out to reveal the main content on the tile. To set a peek image, place an image element in the root of the <binding>, and set the placement attribute to "peek."

XML

```

<binding template="TileMedium" branding="name">
    <image placement="peek" src="Assets/Apps/Hipstame/hipster.jpg"/>
    <text>New Message</text>
    <text hint-style="captionsubtle" hint-wrap="true">Hey, have you tried

```

```
Windows 10 yet?</text>
</binding>
```

C#

```
TileWide = new TileBinding()
{
    Branding = TileBranding.Name,
    Content = new TileBindingContentAdaptive()
    {
        PeekImage = new TilePeekImage()
        {
            Source = "Assets/Apps/Hipstame/hipster.jpg"
        },
        Children =
        {
            new AdaptiveText()
            {
                Text = "New Message"
            },
            new AdaptiveText()
            {
                Text = "Hey, have you tried Windows 10 yet?",
                HintStyle = AdaptiveTextStyle.CaptionSubtle,
                HintWrap = true
            }
        }
    }
}
```

Animated



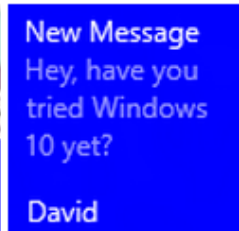
Peek shown



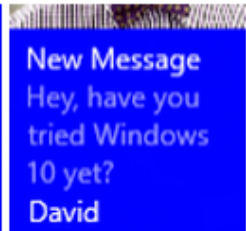
Peek sliding up



Content shown



Peek sliding down



Circle crop for peek and background images

Use the hint-crop attribute on peek and background images to do a circle crop:

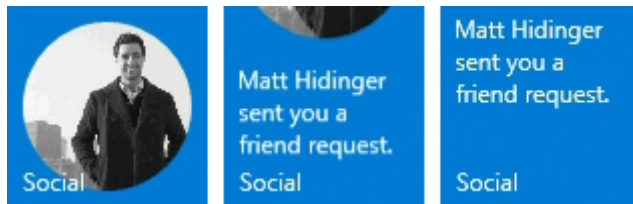
XML

```
<image placement="peek" hint-crop="circle"
src="Assets/Apps/Hipstame/hipster.jpg"/>
```

C#

```
new TilePeekImage()
{
    HintCrop = TilePeekImageCrop.Circle,
    Source = "Assets/Apps/Hipstame/hipster.jpg"
}
```

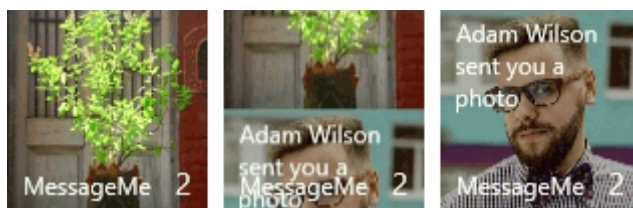
The result will look like this:



Use both peek and background image

To use both a peek and a background image on a tile notification, specify both a peek image and a background image in your notification payload.

The result will look like this:



Peek and background image overlays

You can set a black overlay on your background and peek images using **hint-overlay**, which accepts integers from 0-100, with 0 being no overlay and 100 being full black overlay. You can use the overlay to help ensure that text on your tile is readable.

Use hint-overlay on a background image

Your background image will default to 20% overlay as long as you have some text elements in your payload (otherwise it will default to 0% overlay).

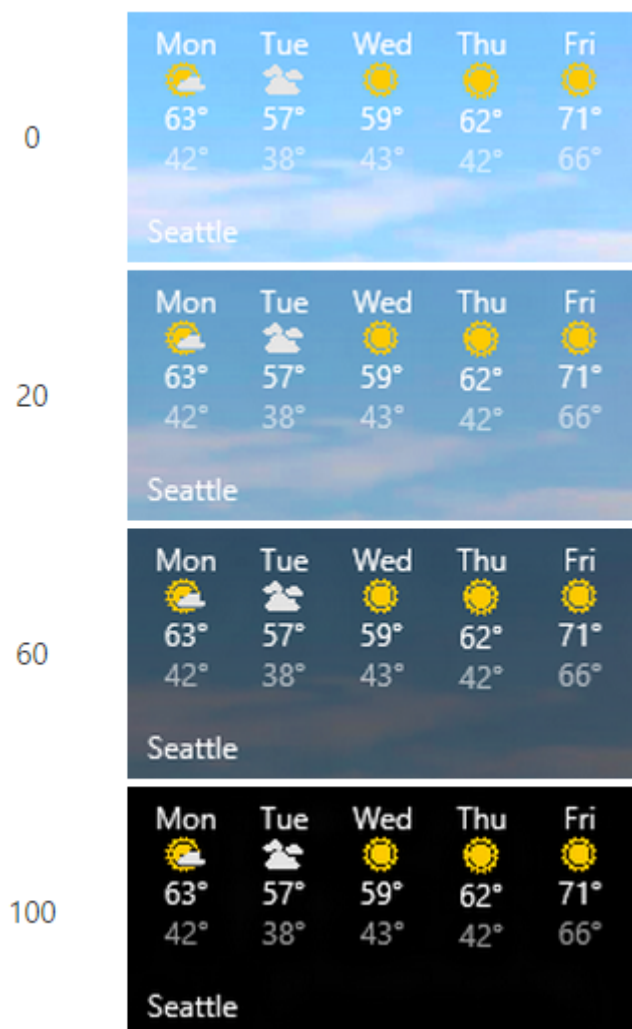
XML

```
<binding template="TileWide">
    <image placement="background" hint-overlay="60" src="Assets\Mostly Cloudy-Background.jpg"/>
    ...
</binding>
```


C#

```
TileWide = new TileBinding()
{
    Content = new TileBindingContentAdaptive()
    {
        BackgroundImage = new TileBackgroundImage()
        {
            Source = "Assets/Mostly Cloudy-Background.jpg",
            HintOverlay = 60
        },
        ...
    }
}
```

hint-overlay Result:



Use hint-overlay on a peek image

Starting in Windows 10 version 1511, we support an overlay for your peek image too, just like your background image. Specify hint-overlay on the peek image element as an integer from 0-100. The default overlay for peek images is 0 (no overlay).

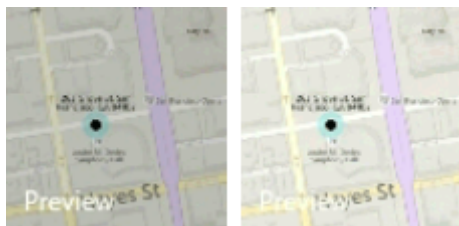
XML

```
<binding template="TileMedium">
  <image hint-overlay="20" src="Assets\Map.jpg" placement="peek"/>
  ...
</binding>
```

C#

```
TileMedium = new TileBinding()
{
    Content = new TileBindingContentAdaptive()
    {
        PeekImage = new TilePeekImage()
        {
            Source = "Assets/Map.jpg",
            HintOverlay = 20
        },
        ...
    }
}
```

This example shows a peek image at 20% opacity (left) and at 0% opacity (right):



Vertical alignment (text stacking)

You can control the vertical alignment of content on your tile by using the `hint-textStacking` attribute on both the `TileBinding` element and `AdaptiveSubgroup` element. By default, everything is vertically aligned to the top, but you can also align content to the bottom or center.

Text stacking on binding element

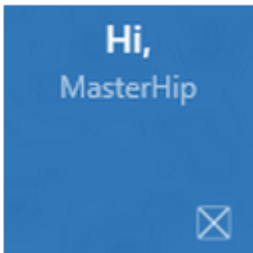
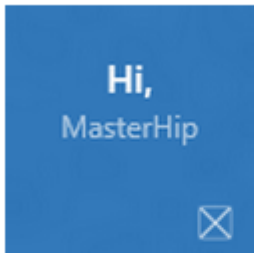
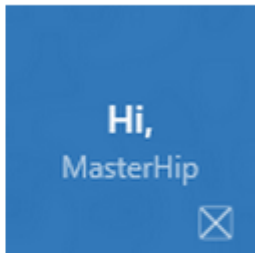
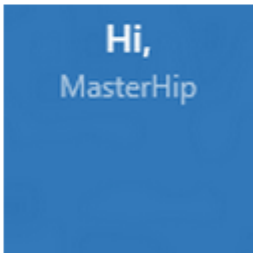
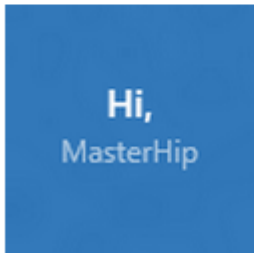
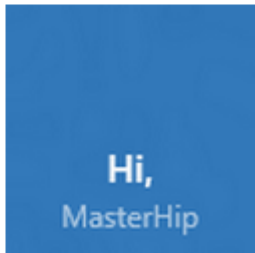
When applied at the `TileBinding` level, text stacking sets the vertical alignment of the notification content as a whole, aligning in the available vertical space above the branding/badge area.

XML

```
<binding template="TileMedium" hint-textStacking="center" branding="logo">
  <text hint-style="base" hint-align="center">Hi,</text>
  <text hint-style="captionSubtle" hint-align="center">MasterHip</text>
</binding>
```

C#

```
TileMedium = new TileBinding()
{
    Branding = TileBranding.Logo,
    Content = new TileBindingContentAdaptive()
    {
        TextStacking = TileTextStacking.Center,
        Children =
        {
            new AdaptiveText()
            {
                Text = "Hi,",
                HintStyle = AdaptiveTextStyle.Base,
                HintAlign = AdaptiveTextAlign.Center
            },
            new AdaptiveText()
            {
                Text = "MasterHip",
                HintStyle = AdaptiveTextStyle.CaptionSubtle,
                HintAlign = AdaptiveTextAlign.Center
            }
        }
    }
}
```

hint-textStacking	top	center	bottom
With branding or badge			
Without branding/badge			

Text stacking on subgroup element


```
new AdaptiveText()  
{  
    Text = "Hi",  
    HintStyle = AdaptiveTextStyle.Subtitle  
},  
  
new AdaptiveText()  
{  
    Text = "MasterHip",  
    HintStyle = AdaptiveTextStyle.BodySubtle  
}  
}  
}  
}  
}  
}  
}  
}  
}
```

Related topics

- [Tile content schema](#)
- [Send a local tile notification](#)
- [Special tile templates](#)
- [UWP Community Toolkit - Notifications](#) 
- [Windows Notifications on GitHub](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Tile content schema

Article • 08/08/2024

 Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

The following describes all of the properties and elements within tile content.

If you would rather use raw XML instead of the [Notifications library](#), please see [the XML schema](#).

- [TileContent](#)
 - [TileVisual](#)
 - [TileBinding](#)
 - [TileBindingContentAdaptive](#)
 - [TileBindingContentIconic](#)
 - [TileBindingContentContact](#)
 - [TileBindingContentPeople](#)
 - [TileBindingContentPhotos](#)

TileContent

TileContent is the top level object that describes a tile notification's content, including visuals.

 Expand table

Property	Type	Required	Description
Visual	ToastVisual	true	Describes the visual portion of the tile notification.

TileVisual

The visual portion of tiles contains the visual specifications for all tile sizes, and more visual-related properties.

Property	Type	Required	Description
TileSmall	TileBinding	false	Provide an optional small binding to specify content for the small tile size.
TileMedium	TileBinding	false	Provide an optional medium binding to specify content for the medium tile size.
TileWide	TileBinding	false	Provide an optional wide binding to specify content for the wide tile size.
TileLarge	TileBinding	false	Provide an optional large binding to specify content for the large tile size.
Branding	TileBranding	false	The form that the tile should use to display the app's brand. By default, inherits branding from the default tile.
DisplayName	string	false	An optional string to override the tile's display name while showing this notification.
Arguments	string	false	New in Anniversary Update: App-defined data that is passed back to your app via the <code>TileActivatedInfo</code> property on <code>LaunchActivatedEventArgs</code> when the user launches your app from the Live Tile. This allows you to know which tile notifications your user saw when they tapped your Live Tile. On devices without the Anniversary Update, this will simply be ignored.
LockDetailedStatus1	string	false	If you specify this, you must also provide a <code>TileWide</code> binding. This is the first line of text that will be displayed on the lock screen if the user has selected your tile as their detailed status app.
LockDetailedStatus2	string	false	If you specify this, you must also provide a <code>TileWide</code> binding. This is the second line of text that will be displayed on the lock screen if the user has selected your tile as their detailed status app.
LockDetailedStatus3	string	false	If you specify this, you must also provide a <code>TileWide</code> binding. This is the third line of text that will be displayed on the lock screen if the user has selected your tile as their detailed status app.

Property	Type	Required	Description
BaseUri	Uri	false	A default base URL that is combined with relative URLs in image source attributes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the toast notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of "www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"
Language	string	false	The target locale of the visual payload when using localized resources, specified as BCP-47 language tags such as "en-US" or "fr-FR". This locale is overridden by any locale specified in binding or text. If not provided, the system locale will be used instead.

TileBinding

The binding object contains the visual content for a specific tile size.

[Expand table](#)

Property	Type	Required	Description
Content	ITileBindingContent	false	The visual content to display on the tile. One of TileBindingContentAdaptive , TileBindingContentIconic , TileBindingContentContact , TileBindingContentPeople , or TileBindingContentPhotos .
Branding	TileBranding	false	The form that the tile should use to display the app's brand. By default, inherits branding from the default tile.

Property	Type	Required	Description
DisplayName	string	false	An optional string to override the tile's display name for this tile size.
Arguments	string	false	New in Anniversary Update: App-defined data that is passed back to your app via the TileActivatedInfo property on LaunchActivatedEventArgs when the user launches your app from the Live Tile. This allows you to know which tile notifications your user saw when they tapped your Live Tile. On devices without the Anniversary Update, this will simply be ignored.
BaseUri	Uri	false	A default base URL that is combined with relative URLs in image source attributes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the toast notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of "www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"
Language	string	false	The target locale of the visual payload when using localized resources, specified as BCP-47 language tags such as "en-US" or "fr-FR". This locale is overridden by any locale specified in binding or text. If not provided, the system locale will be used instead.

ITileBindingContent

Marker interface for tile binding content. These let you choose what you want to specify your tile visuals in - Adaptive, or one of the special templates.

Implementations
TileBindingContentAdaptive
TileBindingContentIconic
TileBindingContentContact
TileBindingContentPeople
TileBindingContentPhotos

TileBindingContentAdaptive

Supported on all sizes. This is the recommended way of specifying your tile content. Adaptive Tile templates new in Windows 10, and you can create a wide variety of custom tiles through adaptive.

 Expand table

Property	Type	Required	Description
Children	<code>ICollection<ITileBindingContentAdaptiveChild></code>	false	The inline visual elements. AdaptiveText , AdaptiveImage , and AdaptiveGroup objects can be added. The children are displayed in a vertical StackPanel fashion.
BackgroundImage	TileBackgroundImage	false	An optional background image that gets displayed behind all the Tile content, full bleed.
PeekImage	TilePeekImage	false	An optional peek image that animates in from the top of the Tile.
TextStacking	TileTextStacking	false	Controls the text stacking (vertical alignment) of the

Property	Type	Required	Description
			children content as a whole.

AdaptiveText

An adaptive text element.

[Expand table](#)

Property	Type	Required	Description
Text	string	false	The text to display.
HintStyle	AdaptiveTextStyle	false	The style controls the text's font size, weight, and opacity.
HintWrap	bool?	false	Set this to true to enable text wrapping. Default to false.
HintMaxLines	int?	false	The maximum number of lines the text element is allowed to display.
HintMinLines	int?	false	The minimum number of lines the text element must display.
HintAlign	AdaptiveTextAlign	false	The horizontal alignment of the text.
Language	string	false	The target locale of the XML payload, specified as a BCP-47 language tags such as "en-US" or "fr-FR". The locale specified here overrides any other specified locale, such as that in binding or visual. If this value is a literal string, this attribute defaults to the user's UI language. If this value is a string reference, this attribute defaults to the locale chosen by Windows Runtime in resolving the string.

AdaptiveTextStyle

Text style controls font size, weight, and opacity. Subtle opacity is 60% opaque.

[Expand table](#)

Value	Meaning
Default	Default value. Style is determined by the renderer.
Caption	Smaller than paragraph font size.
CaptionSubtle	Same as Caption but with subtle opacity.
Body	Paragraph font size.
BodySubtle	Same as Body but with subtle opacity.
Base	Paragraph font size, bold weight. Essentially the bold version of Body.
BaseSubtle	Same as Base but with subtle opacity.
Subtitle	H4 font size.
SubtitleSubtle	Same as Subtitle but with subtle opacity.
Title	H3 font size.
TitleSubtle	Same as Title but with subtle opacity.
TitleNumeral	Same as Title but with top/bottom padding removed.
Subheader	H2 font size.
SubheaderSubtle	Same as Subheader but with subtle opacity.
SubheaderNumeral	Same as Subheader but with top/bottom padding removed.
Header	H1 font size.
HeaderSubtle	Same as Header but with subtle opacity.
HeaderNumeral	Same as Header but with top/bottom padding removed.

AdaptiveTextAlign

Controls the horizontal alignment of text.

 Expand table

Value	Meaning
Default	Default value. Alignment is automatically determined by the renderer.
Auto	Alignment determined by the current language and culture.
Left	Horizontally align the text to the left.

Value	Meaning
Center	Horizontally align the text in the center.
Right	Horizontally align the text to the right.

AdaptiveImage

An inline image.

[Expand table](#)

Property	Type	Required	Description
Source	string	true	The URL to the image. ms-appx, ms-appdata, and http are supported. As of the Fall Creators Update, web images can be up to 3 MB on normal connections and 1 MB on metered connections. On devices not yet running the Fall Creators Update, web images must be no larger than 200 KB.
HintCrop	AdaptiveImageCrop	false	Control the desired cropping of the image.
HintRemoveMargin	bool?	false	By default, images inside groups/subgroups have an 8px margin around them. You can remove this margin by setting this property to true.
HintAlign	AdaptiveImageAlign	false	The horizontal alignment of the image.
AlternateText	string	false	Alternate text describing the image, used for accessibility purposes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the tile notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of

Property	Type	Required	Description
			"www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"

AdaptiveImageCrop

Specifies the desired cropping of the image.

[Expand table](#)

Value	Meaning
Default	Default value. Cropping behavior determined by renderer.
None	Image is not cropped.
Circle	Image is cropped to a circle shape.

AdaptiveImageAlign

Specifies the horizontal alignment for an image.

[Expand table](#)

Value	Meaning
Default	Default value. Alignment behavior determined by renderer.
Stretch	Image stretches to fill available width (and potentially available height too, depending on where the image is placed).
Left	Align the image to the left, displaying the image at its native resolution.
Center	Align the image in the center horizontally, displaying the image at its native resolution.
Right	Align the image to the right, displaying the image at its native resolution.

AdaptiveGroup

Groups semantically identify that the content in the group must either be displayed as a whole, or not displayed if it cannot fit. Groups also allow creating multiple columns.

[Expand table](#)

Property	Type	Required	Description
Children	<code>IList<AdaptiveSubgroup></code>	false	Subgroups are displayed as vertical columns. You must use subgroups to provide any content inside an <code>AdaptiveGroup</code> .

AdaptiveSubgroup

Subgroups are vertical columns that can contain text and images.

[Expand table](#)

Property	Type	Required	Description
Children	<code>IList<IAdaptiveSubgroupChild></code>	false	AdaptiveText and AdaptiveImage are valid children of subgroups.
HintWeight	<code>int?</code>	false	Control the width of this subgroup column by specifying the weight, relative to the other subgroups.
HintTextStacking	AdaptiveSubgroupTextStacking	false	Control the vertical alignment of this subgroup's content.

IAdaptiveSubgroupChild

Marker interface for subgroup children.

[Expand table](#)

Implementations
AdaptiveText
AdaptiveImage

AdaptiveSubgroupTextStacking

TextStacking specifies the vertical alignment of content.

[Expand table](#)

Value	Meaning
Default	Default value. Renderer automatically selects the default vertical alignment.
Top	Vertical align to the top.
Center	Vertical align to the center.
Bottom	Vertical align to the bottom.

TileBackgroundImage

A background image displayed full-bleed on the tile.

[Expand table](#)

Property	Type	Required	Description
Source	string	true	The URL to the image. ms-appx, ms-appdata, and http(s) are supported. Http images must be 200 KB or less in size.
HintOverlay	int?	false	A black overlay on the background image. This value controls the opacity of the black overlay, with 0 being no overlay and 100 being completely black. Defaults to 20.
HintCrop	TileBackgroundImageCrop	false	New in 1511: Specify how you would like the image to be cropped. In versions before 1511, this will be ignored and background image will be displayed without any cropping.
AlternateText	string	false	Alternate text describing the image, used for accessibility purposes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the tile notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified

Property	Type	Required	Description
			without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of "www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"

TileBackgroundImageCrop

Controls the cropping of the background image.

[Expand table](#)

Value	Meaning
Default	Cropping uses the default behavior of the renderer.
None	Image is not cropped, displayed square.
Circle	Image is cropped to a circle.

TilePeekImage

A peek image that animates in from the top of the tile.

[Expand table](#)

Property	Type	Required	Description
Source	string	true	The URL to the image. ms-appx, ms-appdata, and http(s) are supported. Http images must be 200 KB or less in size.
HintOverlay	int?	false	New in 1511: A black overlay on the peek image. This value controls the opacity of the black overlay, with 0 being no overlay and 100 being completely black. Defaults to 20. In previous versions, this value will be ignored and peek image will be displayed with 0 overlay.
HintCrop	TilePeekImageCrop	false	New in 1511: Specify how you would like the image to be cropped. In versions before

Property	Type	Required	Description
			1511, this will be ignored and peek image will be displayed without any cropping.
AlternateText	string	false	Alternate text describing the image, used for accessibility purposes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the tile notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of "www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"

TilePeekImageCrop

Controls the cropping of the peek image.

[Expand table](#)

Value	Meaning
Default	Cropping uses the default behavior of the renderer.
None	Image is not cropped, displayed square.
Circle	Image is cropped to a circle.

TileTextStacking

Text stacking specifies the vertical alignment of the content.

[Expand table](#)

Value	Meaning
Default	Default value. Renderer automatically selects the default vertical alignment.

Value	Meaning
Top	Vertical align to the top.
Center	Vertical align to the center.
Bottom	Vertical align to the bottom.

TileBindingContentIconic

Supported on Small and Medium. Enables an iconic tile template, where you can have an icon and badge display next to each other on the tile, in true classic Windows Phone style. The number next to the icon is achieved through a separate badge notification.

[Expand table](#)

Property	Type	Required	Description
Icon	TileBasicImage	true	At minimum, to support both Desktop and Mobile, Small and Medium tiles, provide a square aspect ratio image with a resolution of 200x200, PNG format, with transparency and no color other than white. For more info see: Special Tile Templates .

TileBindingContentContact

Mobile-only. Supported on Small, Medium, and Wide.

[Expand table](#)

Property	Type	Required	Description
Image	TileBasicImage	true	The image to display.
Text	TileBasicText	false	A line of text that is displayed. Not displayed on small tile.

TileBindingContentPeople

New in 1511: Supported on Medium, Wide, and Large (Desktop and Mobile). Previously this was Mobile-only and only Medium and Wide.

[Expand table](#)

Property	Type	Required	Description
Images	IList< TileBasicImage >	true	Images that will roll around as circles.

TileBindingContentPhotos

Animates through a slideshow of photos. Supported on all sizes.

[Expand table](#)

Property	Type	Required	Description
Images	IList< TileBasicImage >	true	Up to 12 images can be provided (Mobile will only display up to 9), which will be used for the slideshow. Adding more than 12 will throw an exception.

TileBasicImage

An image used on various special templates.

[Expand table](#)

Property	Type	Required	Description
Source	string	true	The URL to the image. ms-appx, ms-appdata, and http(s) are supported. Http images must be 200 KB or less in size.
AlternateText	string	false	Alternate text describing the image, used for accessibility purposes.
AddImageQuery	bool?	false	Set to "true" to allow Windows to append a query string to the image URL supplied in the tile notification. Use this attribute if your server hosts images and can handle query strings, either by retrieving an image variant based on the query strings or by ignoring the query string and returning the image as specified without the query string. This query string specifies scale, contrast setting, and language; for instance, a value of "www.website.com/images/hello.png" given in the notification becomes "www.website.com/images/hello.png?ms-scale=100&ms-contrast=standard&ms-lang=en-us"

TileBasicText

A basic text element used on various special templates.

 Expand table

Property	Type	Required	Description
Text	string	false	The text to display.
Language	string	false	The target locale of the XML payload, specified as a BCP-47 language tags such as "en-US" or "fr-FR". The locale specified here overrides any other specified locale, such as that in binding or visual. If this value is a literal string, this attribute defaults to the user's UI language. If this value is a string reference, this attribute defaults to the locale chosen by Windows Runtime in resolving the string.

Related topics

- [Send a local tile notification](#)
- [Notifications library on GitHub](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Special tile templates

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Special tile templates are unique templates that are either animated, or just allow you to do things that aren't possible with adaptive tiles. Each special tile template was specifically built for Windows 10, except for the iconic tile template, a classic special template that has been updated for Windows 10. This article covers three special tile templates: Iconic, Photos, and People.

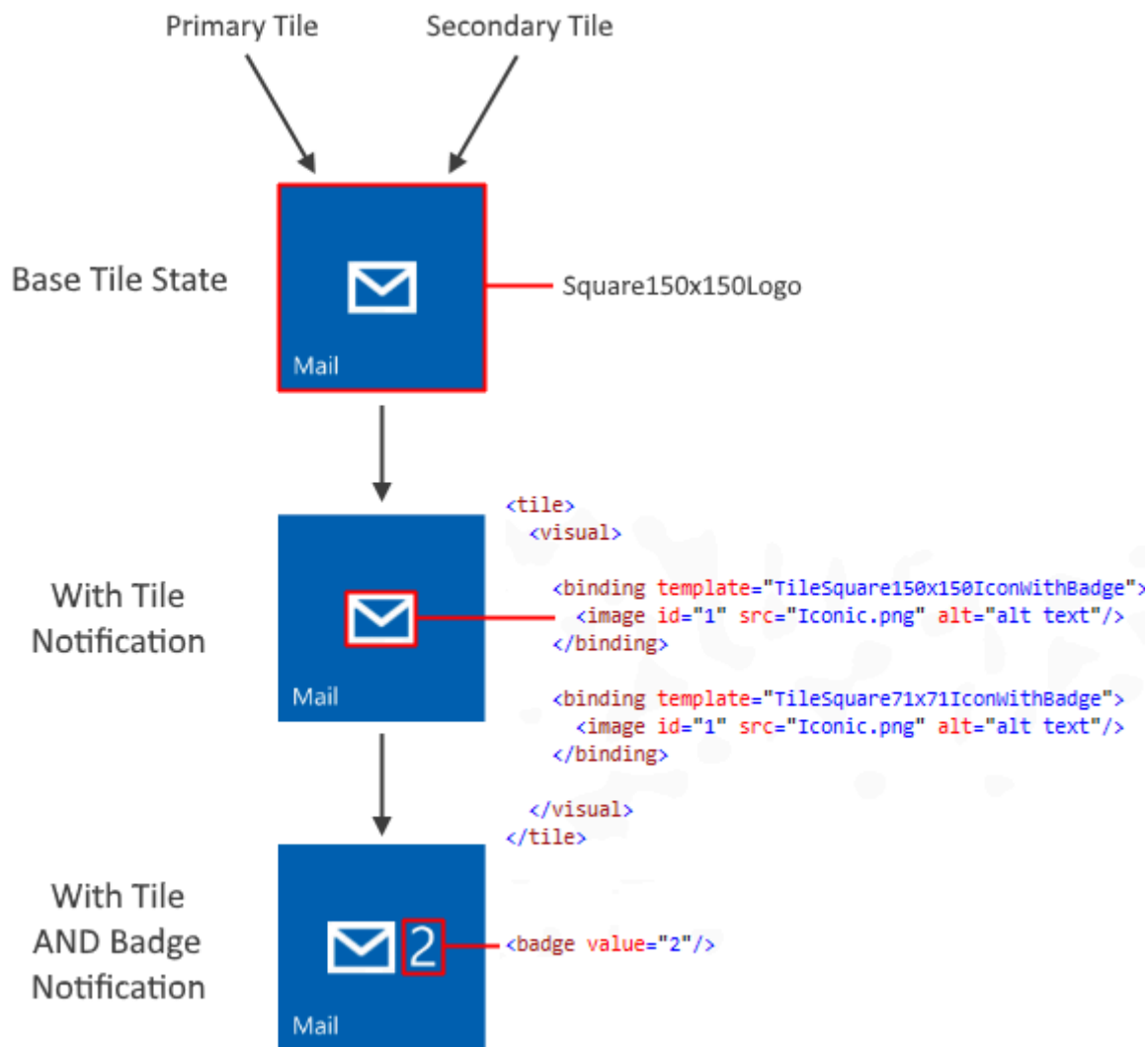
Iconic tile template

The iconic template (also known as the "IconWithBadge" template) lets you display a small image in the center of the tile.



How to create an iconic tile

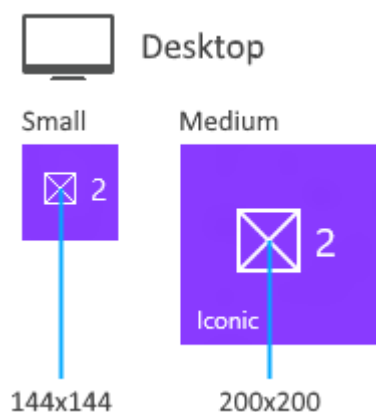
The following steps cover everything you need to know to create an iconic tile for Windows 10. At a high level, you need your iconic image asset, then you send a notification to the tile using the iconic template, and finally you send a badge notification that provides the number to be displayed on the tile.



Step 1: Create your image assets in PNG format

Create the icon assets for your tile and place those in your project resources with your other assets. At a bare minimum, create a 200x200 pixel icon, which works for both small and medium tiles. To provide the best user experience, create an icon for each size. No padding is needed on these assets. See sizing details in the below image. Windows 10 on tablet, laptop, and desktop only supports square icon assets.

Save icon assets in PNG format and with transparency.



Step 2: Create your base tile

You can use the iconic template on both primary and secondary tiles. If you're using it on a secondary tile, you'll first have to create the secondary tile or use an already-pinned secondary tile. Primary tiles are implicitly pinned and can always be sent notifications.

Step 3: Send a notification to your tile

Although this step can vary based on whether the notification is sent locally or via server push, the XML payload that you send remains the same. To send a local tile notification, create a [TileUpdater](#) for your tile (either primary or secondary tile), then send a notification to the tile that uses the iconic tile template as seen below. Ideally, you should also include bindings for wide and large tile sizes using [adaptive tile templates](#).

Here's sample code for the XML payload:

XML

```
<tile>
  <visual>

    <binding template="TileSquare150x150IconWithBadge">
      <image id="1" src="Iconic.png" alt="alt text"/>
    </binding>

    <binding template="TileSquare71x71IconWithBadge">
      <image id="1" src="Iconic.png" alt="alt text"/>
    </binding>

  </visual>
</tile>
```

This iconic tile template XML payload uses an image element that points to the image that you created in Step 1. Now your tile is ready to display the badge next to your icon; all that's left is sending badge notifications.

Step 4: Send a badge notification to your tile

As with step 3, this step can vary based on whether the notification is sent locally or via server push, yet the XML payload that you send remains the same. To send a local badge notification, create a [BadgeUpdater](#) for your tile (either primary or secondary tile), then send a badge notification with your desired value (or clear the badge).

Here's sample code for the XML payload:

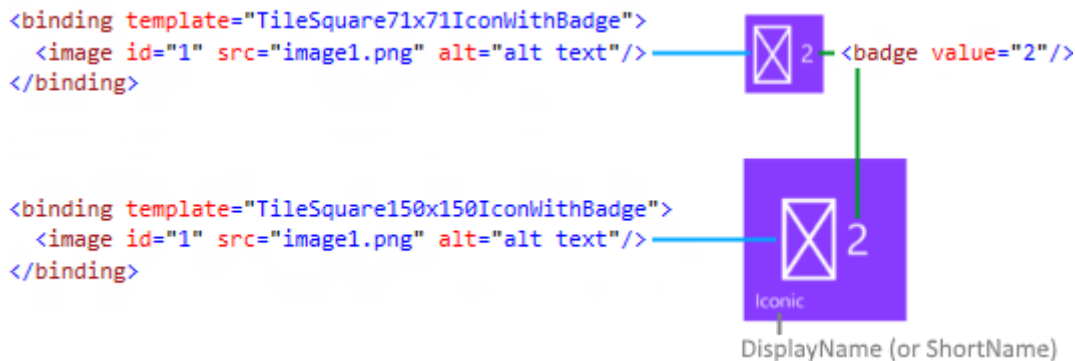
XML


```
<badge value="2"/>
```

The tile's badge will update accordingly.

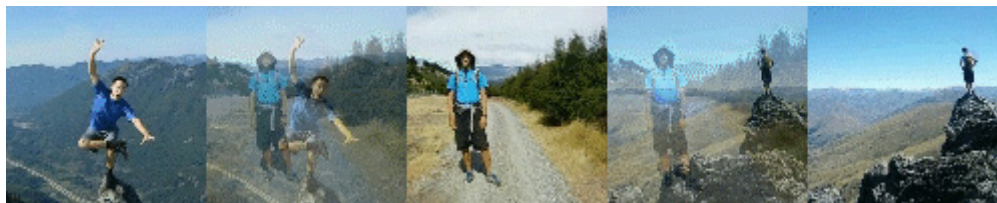
Step 5: Putting it all together

The following image illustrates how the various APIs and payloads are associated with each aspect of the iconic tile template. A [tile notification](#) (which contains those `<binding>` elements) is used to specify the iconic template and the image asset; a [badge notification](#) specifies the numerical value; tile properties control your tile's display name, color, and more.



Photos tile template

The photos tile template lets you display a slideshow of photos on your live tile. The template is supported on all tile sizes, including small, and behaves the same on each tile size. The below example shows five frames of a medium tile that uses the photos template. The template has a zoom and cross-fade animation that cycles through selected photos and loops indefinitely.



How to use the photos template

Using the photos template is easy if you've installed the [Notifications library](#). Although you can use raw XML, we highly recommend using the library so you don't have to worry about generating valid XML or XML-escaping content.

Windows displays up to 12 photos in a slideshow.

For information about sending the tile notification, see the [Send notifications article](#).

XML

```
<!--
```

```
To use the Photos template...
```

- On any adaptive tile binding (like TileMedium or TileWide)
 - Set the hint-presentation attribute to "photos"
 - Add up to 12 images as children of the binding.

```
-->
```

```
<tile>
```

```
  <visual>
```

```
    <binding template="TileMedium" hint-presentation="photos">
```

```
      <image src="Assets/1.jpg" />
```

```
      <image src="ms-appdata:///local/Images/2.jpg"/>
```

```
      <image src="http://msn.com/images/3.jpg"/>
```

```
      <!--TODO: Can have 12 images total-->
```

```
    </binding>
```

```
    <!--TODO: Add bindings for other tile sizes-->
```

```
  </visual>
```

```
</tile>
```

C#

```
/*
```

```
To use the Photos template...
```

- On any TileBinding object
 - Set Content property to new instance of TileBindingContentPhotos
 - Add up to 12 images to Images property of TileBindingContentPhotos.

```
*/
```

```
TileContent content = new TileContent()
```

```
{
```

```
    Visual = new TileVisual()
```

```
    {
```

```
        TileMedium = new TileBinding()
```

```
        {
```

```
            Content = new TileBindingContentPhotos()
```

```
            {
```

```
                Images =
```

```
                {
```

```
                    new TileBasicImage() { Source = "Assets/1.jpg" },
```

```

        new TileBasicImage() { Source = "ms-
appdata:///local/Images/2.jpg" },
        new TileBasicImage() { Source =
"http://msn.com/images/3.jpg" }

        // TODO: Can have 12 images total
    }
}

// TODO: Add other tile sizes
}
};

```

People tile template

The People app in Windows 10 uses a special tile template that displays a collection of images in circles that slide around vertically or horizontally on the tile. This tile template has been available since Windows 10 Build 10572, and anyone is welcome to use it in their app.

The People tile template works on tiles of these sizes:

Medium tile (TileMedium)



Wide tile (TileWide)



Large tile (desktop only) (TileLarge)



If you're using the [Notifications library](#), all you have to do to make use of the People tile template is create a new *TileBindingContentPeople* object for your *TileBinding* content. The *TileBindingContentPeople* class has an *Images* property where you add your images.

If you're using raw XML, set the *hint-presentation* to "people" and add your images as children of the binding element.

The following C# code sample assumes that you're using the [Notifications library](#).

C#

```
TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        TileMedium = new TileBinding()
        {
            Content = new TileBindingContentPeople()
            {
                Images =
                {
                    new TileBasicImage() { Source =
"Assets/ProfilePics/1.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/2.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/3.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/4.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/5.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/6.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/7.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/8.jpg" },
                    new TileBasicImage() { Source =
"Assets/ProfilePics/9.jpg" }
                }
            }
        }
    }
}
```

```
};  
    }  
    }  
}
```

XML

```
<tile>  
  <visual>  
  
    <binding template="TileMedium" hint-presentation="people">  
      <image src="Assets/ProfilePics/1.jpg" />  
      <image src="Assets/ProfilePics/2.jpg" />  
      <image src="Assets/ProfilePics/3.jpg" />  
      <image src="Assets/ProfilePics/4.jpg" />  
      <image src="Assets/ProfilePics/5.jpg" />  
      <image src="Assets/ProfilePics/6.jpg" />  
      <image src="Assets/ProfilePics/7.jpg" />  
      <image src="Assets/ProfilePics/8.jpg" />  
      <image src="Assets/ProfilePics/9.jpg" />  
    </binding>  
  
  </visual>  
</tile>
```

For the best user experience, we recommend that you provide the following number of photos for each tile size:

- Medium tile: 9 photos
- Wide tile: 15 photos
- Large tile: 20 photos

Having that number of photos allows for a few empty circles, which means that the tile won't be too visually busy. Feel free to tweak the number of photos to get the look that works best for you.

To send the notification, see [Choose a notification delivery method](#).

Related topics

- [Full code sample on GitHub](#) 
- [Notifications library](#) 
- [Create adaptive tiles](#)
- [Tile content schema](#)